

Spec-Driven Development · Hands-on session

From an ambiguous brief to an executable specification

Emilio Delgado Muñoz

University Institutes for Applied Computing Technologies
Universidad de Extremadura · edm@unex.es

Extremadura Digital Day (EDD) · 2026

using Spec Kit v1.0.2 and Claude Code via OpenRouter

Before we start: what we are NOT doing today

- **We are not giving a theory talk.** Five minutes of framing and then hands on the keyboard.
- **We are not going to finish an application.** That is impossible in 90 minutes and it is not the point.
- **We are not going to measure** how much code the agent produces.

What we are doing instead

Running the full cycle on a small problem and finding out, hands on, where the real difference lies between this and just asking an agent for code.

Goals

By the end you should be able to:

1. Initialise a Spec Kit project and explain what lives inside `.specify/`.
2. Write a constitution with *verifiable* principles.
3. Review a generated specification critically: spot decisions the agent made and you did not.
4. Resolve ambiguities explicitly.
5. Derive a plan and tasks, and detect inconsistencies between artefacts.
6. Absorb a requirement change by editing the specification, not the code.

The idea behind the whole session

*The valuable artefact is the specification.
The code is a regenerable consequence of it.*

What these 90 minutes are for

By the end you should have gone through a complete development—including a requirement change—driven exclusively by the specification.

Min	Block	Content
0–5	Warm-up	Environment check (installed at home beforehand)
5–12	Framing	SDD in five minutes and the PenaLib case
12–25	A	init and constitution
25–45	A	specify, critical reading and clarify
45–58	B	plan, tasks and analyze
58–72	B	implement of the first version
72–85	C	The requirement change
85–90	Wrap-up	Discussion, limits and criticism

Blocks A, B and C are your work. I talk very little (which is not actually true).

How the workshop works

Purple: your work

Exercises, many of them in pairs. This is the real content of the session.

Green: verification

Do not move on until it holds.

Orange: watch out

Places where people get stuck.

Blue: context

Explanation, no action required.

Everything is in the handbook too, in more detail. If you get lost, go over it afterwards.

Warm-up: environment check

min. 0–5

Raise your hand if any of these five fails:

1. `python3 --version` responds (3.11 or later).
2. `git --version` responds.
3. `uv --version` responds (it is Astral's package manager).
4. `claude --version` responds.
5. `specify --help` responds.

This was supposed to be done at home

The setup guide went out with the invitation. Today we only check it. If something fails, raise your hand: you can always work in pairs.

Reminder: Claude Code against OpenRouter

The four lines you should already have in your `~/.bashrc` or `~/.zshrc`:

```
export OPENROUTER_API_KEY="sk-or-..."
export ANTHROPIC_BASE_URL="https://openrouter.ai/api"
export ANTHROPIC_AUTH_TOKEN="$OPENROUTER_API_KEY"
export ANTHROPIC_API_KEY="" # essential: empty
```

Three warnings

Line 1 carries *your* OpenRouter key, the one you set up before the workshop.

Line 4 must stay empty; otherwise Claude tries to authenticate against Anthropic.

Your key is money and it is personal: it must not end up in the repository.

Support file: `bashrc-openrouter.txt` at tinyurl.com/sdd-edd

End-to-end check

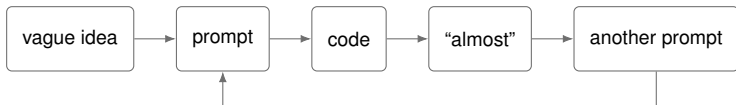
1. `specify self check` responds and tells you the latest available version.
2. Run `claude`.
3. Inside Claude, `/status` shows `ANTHROPIC_AUTH_TOKEN` and the OpenRouter URL (not an Anthropic account).
4. Your first request shows up in the OpenRouter activity dashboard. That is where you will watch the spend all session long.
5. Leave Claude with `/exit`.

Checkpoint 0

`/status` points at OpenRouter, `specify` responds and you have credit. If this works, the rest will work.

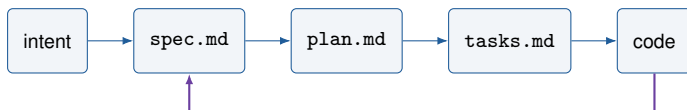
SDD in five minutes, and the case study

min. 5–12



- Business decisions end up buried in the code.
- The conversation with the agent is lost.
- Nobody can review *what was decided*, only *what was written*.

Today's loop



the change comes back here, not to the code

- Each phase produces a document that the next one consumes.
- Everything is version-controlled Markdown, reviewable in a pull request.
- Code stops being the source of truth.

What SDD is not

- **It is not documentation.** Documentation describes what was already done; the specification is the *input* to the process.
- **It is not magic.** It does not remove work: it moves it.
- **It is not deterministic.** Two people with the same brief will get different specs. We will see that live today.
- **It is not free.** It burns tokens and time. By the end of the session you will have real figures for your own spend.

The brief, exactly as it arrives

```
The school library lends books to its members. Every loan has  
a checkout date and a due date. When a member returns a book  
later than the due date, a penalty is generated. A member with  
outstanding penalties cannot borrow any more books. We need a  
tool that records loans and returns and computes the penalties.
```

Four lines. This is exactly what the person running the library would tell you...
and exactly what many people would hand to an agent before saying “build me the app”.

A question for the room

How many business decisions are left unresolved in that brief?

(15 seconds. Shout out a number.)

Ten, at the very least

1. Calendar days or working days? Holidays?
2. How much is the penalty?
3. Is there a cap?
4. What does “outstanding” mean?
5. Can a loan be renewed?
6. How many books at once?
7. Does returning on the due date count?
8. What if the book is lost?
9. Are all members the same?
10. Who waives a penalty? Is it recorded?

The problem is not that they are missing

An agent with no specification will answer all ten on its own, silently and plausibly... and you will find out once the system is in production.

Block A: from an empty project to the specification

min. 12–45

Block A: what we are going to do

1. Initialise the project and the repository.
2. Write the constitution. (There is a trap in it.)
3. Generate the specification from the library brief.
4. Read it critically. **This is 60 % of the value of the session.**
5. Resolve the ambiguities with `/speckit-clarify`.

```
specify init penalib --integration claude
cd penalib

# Spec Kit no longer creates the repository: do it by hand
git init
specify extension add git

# first commit, so block C has something to compare against
git add -A
git commit -m "chore: Spec Kit scaffolding"

git status
claude
```

Support file: `init.txt` at tinyurl.com/sdd-edd

The repository does not come ready made

It changed in v0.10.0

Up to that version, `specify init` initialised the repository. Now the `git` extension is optional and you have to ask for it: a `git status` right after `init` tells you there is no repository there.

- Without that `git init` you lose half the session: the diff exercise in block C cannot be done.
- The extension also ships `/speckit-git-initialize`, which does the same from inside the agent.

Note what just happened: a tutorial written against an earlier version breaks on the very first command. That is why we pin version 1.0.2.

Three Claude Code shortcuts for today

`Esc` Interrupts the agent mid-answer. You will use it in block B.

`/clear` Empties the conversation context. Worth doing between long phases: the artefacts live in files, so you lose nothing.

`/help` Lists the available commands, including the project's own.

Checkpoint A.1

`.specify/` and `.claude/skills/` exist; `git status` answers as a repository; `git log` shows your initial commit; `/speckit-` autocompletes.

The constitution

- It is not a manifesto: it is a set of *constraints* the agent reads every time it plans or implements.
- It lives in `.specify/memory/constitution.md`.
- It can be edited by hand. It is ordinary Markdown.

Short beats long

With five well-chosen principles, the agent respects them. With twenty, the context is diluted and it starts ignoring them selectively. Exactly like human style guides.

The constitution prompt

```
/speckit-constitution Create the principles for a small
Python command-line project maintained by students.
Priorities: (1) every business rule must be covered by at
least one automated test with an edge case; (2) amount and
date calculations must be deterministic and must not depend
on the machine's time zone; (3) no external dependencies
other than the test framework; (4) any visible behaviour
change requires updating the specification before the code;
(5) the code must be clean and readable.
```

Support file: `constitution.en.txt` at tinyurl.com/sdd-edd

Then open `.specify/memory/constitution.md` and read it.

The dot and the hyphen

You will trip over this

The official documentation writes `/speckit.constitution`, with a dot. The Claude integration installs *skills*, whose separator is the hyphen: in Claude Code you type `/speckit-constitution`.

- Other agents use `$speckit-constitution` or `/skill:speckit-constitution`.
- Some of Spec Kit's own error messages still print the dotted form. It is a known defect: translate it in your head.
- The files live in `.claude/skills/`. You can open and read them.

Exercise A.1

Verifiable principles · 4 minutes, in pairs

We slipped in a bogus principle on purpose.

The first four can be checked: either there is a test with an edge case, or there is not. The fifth (“the code must be clean and readable”) cannot be checked and therefore constrains nothing.

Rewrite principle 5 to make it verifiable, or delete it. Edit the file by hand.

Quick round: what did you write?

We describe *what* we want. No language, no database, no files.

```
/speckit-specify The school library lends books to its  
members. Every loan has a checkout date and a due date.  
When a member returns a book later than the due date, a  
penalty is generated. A member with outstanding penalties  
cannot borrow any more books. We need a tool that records  
loans and returns and computes the penalties.
```

Support file: `specify.en.txt` at tinyurl.com/sdd-edd

More files will appear than you expect

File	Created by	What it holds
spec.md	specify	the what and the why
checklists/requirements.md	specify, clarify	requirements quality; implement uses it as a gate
plan.md	plan	stack, architecture, decisions
research.md	plan	alternatives considered and discarded
data-model.md	plan	entities, fields, validations
contracts/	plan	interfaces: signatures, commands, schemas
quickstart.md	plan	how to run and validate by hand
tasks.md	tasks	tasks ordered by dependency

It is not noise: each phase writes down its own reasoning so it can be reviewed separately.

A rule to navigate the zoo of files

Look at who wrote it and in which phase.

- Written by `specify` → it is a business decision, and it should contain no technology.
- Written by `plan` → it is a technical decision.
- They are all Markdown and they are all editable by hand. If something is wrong, fix it in the file where it originates.
- The exact set varies with the feature and the version. Do not panic.

What just happened

- A numbered directory under `specs/`, with `spec.md`.
- Next to it, `checklists/requirements.md`: the built-in requirements quality checklist.
- If you added the git extension, a branch for the feature as well.

*Now the real work starts.
And it is not the agent doing it.*

Exercise A.2 · Critical reading

8 minutes, in pairs · write the answers down

1. Which business decisions did the agent make that you had not made?
2. Did it leave explicit ambiguity markers? Where?
3. Is there any requirement you would not know how to verify?
4. Is there any acceptance criterion without numbers?
5. Has any technical decision crept in that has no business being here?

Keep the answers: you will use them in the next block. The other five questions (invented functionality, missing edge cases, readability for a business reader, contradictions with the constitution, project continuity) are in the handbook, section 5.3, for afterwards.

Exercise A.3 · Compare with the pair next to you

2 minutes

You started from the same brief. Find **one** business rule where your `spec.md` and your neighbours' say different things.

What you have just demonstrated

That divergence is not a defect of the tool: it is experimental proof that an ambiguous brief does not determine a system.

The difference between doing this and doing nothing is that here the divergence is written in a file that can be reviewed, instead of buried in an `if` on line 240.

Resolve the ambiguities

`/speckit-clarify`

- It asks up to five questions per pass, about the under-specified areas.
- It can be run several times, each focused on a different area.
- It is the closest thing to a requirements interview you will get today.

Answer with judgement

Every answer is folded into the specification and will condition the plan, the tasks and the code. Do not answer the first thing that comes to mind.

Decisions for the workshop

Decision	Value
Unit of lateness	Whole calendar days, holidays not excluded
Amount	0.20 € per day late and per book
Cap	20.00 € per loan
Outstanding penalty	One that has been neither paid nor waived
Loan limit	3 books at a time per member
Renewals	One of 14 days, only with no outstanding penalties
Return on the due date	Not late, whatever the hour

You may use your own if you can argue for them. Then compare `spec.md` with your answers to exercise A.2.

Closing block A

Checkpoint A.2 · before moving to the plan

- No ambiguity markers are left in `spec.md`.
- Every amount, deadline and limit has a concrete number.
- The specification still mentions no language, no files and no libraries.
- You have committed.

The specification is source code. Treat it as such.

Block B: plan, tasks and implementation

min. 45–72

Now, and only now, technology gets decided. The whole room uses the same stack:

```
/speckit-plan A command-line application in Python 3.11 using  
the standard library only, plus pytest. Persistence is a  
single JSON file in the working directory. No database, no  
server, no web interface. Dates are handled with date, never  
with datetime. The penalty calculation logic must live in a  
pure module, with no input/output, so it can be tested in  
isolation.
```

Support file: `plan.en.txt` at tinyurl.com/sdd-edd

The full cycle

Phase	Command	Produces
Principles	<code>/speckit-constitution</code>	<code>constitution.md</code>
What and why	<code>/speckit-specify</code>	<code>spec.md</code> + requirements checklist
Ambiguities	<code>/speckit-clarify</code>	<code>spec.md</code> with no gaps
How	<code>/speckit-plan</code>	<code>plan.md</code> and design artefacts
Quality	<code>/speckit-checklist</code>	checklists
Order	<code>/speckit-tasks</code>	<code>tasks.md</code>
Consistency	<code>/speckit-analyze</code>	report (read only)
Build	<code>/speckit-implement</code>	code and tests
Close	<code>/speckit-converge</code>	the tasks that were missing

Optional does not mean dispensable



What separates SDD from a tidy chat with an agent

`clarify`, `checklist` and `analyze` are precisely the commands almost nobody uses when trying the tool on their own. They are the ones that carry the most weight today.

Every artefact has a different reader

Artefact	Answers	Who should be able to review it
<code>constitution.md</code>	Under which rules do we work?	The team, once
<code>spec.md</code>	What must it do and why?	Someone from the business, non-technical
<code>plan.md</code>	With which technology?	A technical person
<code>tasks.md</code>	In which order?	Whoever implements, human or agent
Code	—	It gets reviewed, but it no longer rules

If your `spec.md` talks about database indexes, it is badly written.

How much it asks you: two different things

Requirements questions

Spec Kit decides these, and you control them by choosing a route:

- **Short:** specify → plan → tasks → implement. Barely asks anything.
- **Full:** + clarify, checklist, analyze. clarify asks up to 5 questions per pass.

Permissions

Claude Code decides these. **Shift+Tab** to switch:

- **Manual:** asks before almost everything.
- **Accept edits:** edits without asking.
- **Plan:** explores, does not modify.
- **Auto:** another model reviews for you.
- **Bypass permissions:** no checks at all.

Today we take the full route. The questions are the content.

Which mode to use today

Recommendation

Manual while you generate artefacts (block A and plan/tasks).

Accept edits while you implement.

- Automatic mode is reasonable when you already know what you are building and mistakes are cheap.
- It is a bad idea when the goal is to learn what the agent is deciding.
- General rule outside the classroom: the less reversible the action, the less automation. Editing a version-controlled file is reversible; running commands against a real system is not.

What to review in the plan

Several files are generated, not just `plan.md`: `research.md`, `data-model.md`, `contracts/`, `quickstart.md`. Skim them.

Then, two questions:

1. Does the plan respect the constitution? If you banned external dependencies, none should appear.
2. Does the plan cover every requirement in the specification, or has one been dropped along the way?

Tasks and cross-review

`/speckit-tasks`

A dependency-ordered list, grouped into phases. Check whether the tests come before or after each rule: that is a good indicator of whether your constitution is having any real effect.

`/speckit-analyze`

Consistency and coverage analysis across `spec.md`, `plan.md` and `tasks.md`. It is read only: it touches nothing and produces a report.

Exercise B.1 · The finding

4 minutes

Write down one concrete problem detected by analyze and classify it: is it a fault in the specification, in the plan or in the tasks?

Fix it **in the artefact where the problem originates**, not in the one where it shows up.

*Fix the source, not the symptom.
This reflex is the core skill of the session.*

Do not ask for the whole application

Here we implement only the penalty calculation module and its tests. If you let the agent loose, it will spend twenty minutes generating code we are not going to look at and burn your credit.

If it runs away: Esc and rephrase. Interrupting is cheap; undoing is not.

Good moment to switch Claude to *accept edits* with Shift+Tab so it stops asking you thirty times.

Before implementing: consolidate in git

```
/speckit-git-commit
```

The agent will offer it to you: the git extension registers a hook on the `before_implement` event.

Why exactly here

This is the last point in the cycle where everything in the repository is decisions: specification, plan and tasks, without a single line of code.

If you commit now, the next commit will contain exactly what the implementation produced, and the history reads as “this is what we decided / this is what was built from it”. If you do not, the two get mixed and you lose that separation. . . which is exactly the one you need in exercise C.1.

Implement

```
/speckit-implement Implement only the tasks related to the  
pure penalty calculation module and its tests. Do not  
implement persistence or the command-line interface yet.
```

Support file: `implement.en.txt` at tinyurl.com/sdd-edd

B.2 · Observation sheet, while the agent works

- On which task did it get stuck or have to redo something?
- Can you tell it has read the constitution? Where, specifically?
- Did it introduce any rule that is not in the specification?
- Do the tests cover the edge cases from exercise A.2?

Exercise B.3 · What did this cost

1 minute

Open the OpenRouter activity dashboard and note the cumulative cost of the session and that of the implementation phase separately.

We will compare it with the cost of absorbing a requirement change, so that in the final discussion you have real figures instead of intuitions.

Then, the tests:

```
python3 -m pytest -q
```

Closing block B

Checkpoint B.1

The tests pass, there is a calculation module with no input/output, and you can point at the line in the code that implements the 20 € cap and the one that implements the 0.20 € amount.

If you cannot point at them, you have not reviewed the code. Do it now: you are going to need it in the next block.

Block C: the requirement change

min. 72–85

Without this block, everything before it is just an elaborate way to ask for code

Now the rules change.

From next year on we distinguish two kinds of member. Students get three days of grace before the penalty starts counting, and a cap of 10 euros. Faculty members incur no penalties, but they cannot have more than five loans at a time. On top of that, library staff must be able to waive a penalty stating a mandatory reason, and there must be a record of who waived it and when.

Support file: `specify2.en.txt` at tinyurl.com/sdd-edd

What this change breaks

- A new attribute in the model: the member type.
- Two divergent calculation rules where there used to be one.
- A limit that depends on the member type.
- An audit operation that did not exist before.

In a conventional development this is several hours of touching code and discovering side effects.

First decision: where does the amendment live?

Option	When it makes sense	What it costs
A. Amend 001	The change redefines rules that were already specified: amount, cap, who gets penalised. The spec must describe the system as it is today.	A single, readable spec, but “what we decided and when” lives only in the git history.
B. Create 002	The change adds a separable capability with its own delivery: e.g. the audited waiver.	Two specs talk about the same domain and can contradict each other. You must write down which one rules.
C. Mixed	What a real team would do: amend 001 and open 002 for the waiver.	More ceremony and two branches to coordinate.

A detail that surprises everybody

Switching branches does not switch feature

Spec Kit does not infer the active feature from the git branch: it reads it from `.specify/feature.json` (or from `SPECIFY_FEATURE_DIRECTORY`).

A `git checkout` on its own does not change what the commands target.

For the workshop: option A, amend 001.

It is the one that makes the diff exercise possible.

The right procedure

Note what we are *not* going to do: we are not opening the calculation module.

1. `/speckit-specify` — update the specification. That is where the change originates.
2. `/speckit-clarify` — do the three grace days count towards the loan limit or only towards the amount? What happens to penalties already generated for people who are now faculty?
3. `/speckit-plan` and `/speckit-tasks` — regenerate.
4. `/speckit-analyze` — check consistency.
5. `/speckit-implement` — bounding the scope to the calculation module again.

Exercise C.1 · The diff that matters

```
git diff --stat HEAD~1  
git diff HEAD~1 -- specs/
```

6 minutes · one sentence per question

- How many lines changed in `specs/` and how many in the code?
- Reading only the specification diff, is it clear what was decided and why?
- If someone asks a year from now “why do faculty members get no fines?”, where is the answer?

The point to take home

Traditional flow

- The change is applied to the code.
- The documentation silently falls behind.
- The reason for the decision is lost.

Today's flow

- The change is applied to the document.
- The code is derived from it.
- The reason stays in a reviewable diff.

*It is not magic and it does not remove work:
it changes where the truth of the system lives.*

Wrap-up and discussion

min. 85–90

1. In what kind of project does this process *not* pay off? Be specific.
2. Specifications age and accumulate debt too. What stops `spec.md` from becoming the new unreadable legacy?
3. Who should write the specification in a real team: whoever develops, whoever defines the product, both?
4. How would you review a specification in a pull request? What would you look for?
5. Look at the OpenRouter dashboard. With those figures in front of you, when is this process worth it and when is it not?
6. If the model produces a plausible but wrong specification, what mechanism catches it before production?

The criticisms worth taking seriously

- **Cost.** The full cycle burns many more tokens than just asking for code. For a twenty-line script it is absurd.
- **Ceremony.** There are projects where maintaining the artefacts costs more than they are worth.
- **False sense of rigour.** A well-written specification can be completely wrong. Format does not validate content.
- **New debt.** Forty stale specs are as bad as forty undocumented modules.
- **Dependence** on a young tool that breaks its interface every few versions. You have suffered that today.

What to take away, even if you never use Spec Kit again

1. An ambiguous brief does not determine a system. Somebody will decide what is missing: better that it be explicit.
2. If a requirement has no number, it is not a requirement yet.
3. The place to fix a problem is where it originates, not where it shows up.
4. Separate the *what* from the *how*, and give each the reviewer it deserves.
5. Facing a change, go to the highest affected artefact and regenerate downwards.

Taking the work home

The `penalib` project is yours. It is worth making sure it contains:

1. `constitution.md`, with principle 5 fixed.
2. `specs/<NNN>-.../spec.md` in its final version, after the change.
3. `plan.md` and `tasks.md`.
4. The calculation module and its tests, with `pytest` green.

Before sharing the repository with anyone

Check that your OpenRouter key is not travelling inside it:

```
git log -p | grep sk-or-
```

If it shows up, revoke it from the OpenRouter dashboard.

Where to go next

- Repository: `github.com/github/spec-kit`
- Documentation: `github.github.io/spec-kit`
- Full methodology: the `spec-driven.md` file in the repository
- Claude Code: `code.claude.com/docs`
- Customisation: extensions, presets and bundles (`specify extension`, `specify preset`, `specify bundle`)
- Existing projects: the brownfield flow and `/speckit-converge`

When looking for tutorials

Almost everything published before mid-2026 uses the old syntax and assumes `specify init` creates the repository. Always check against the documentation for your version.

Thank you

Emilio Delgado Muñoz

University Institutes for Applied Computing Technologies

Universidad de Extremadura · edm@unex.es

Material adapted from the original workshop by Gregorio Robles (URJC),

gsyc.urjc.es/grex/sdd

Backup slides

Common problems (1/2)

Symptom	What to do
<code>git status</code> says it is not a repository	Normal since v0.10.0. <code>git init</code> and <code>specify extension add git</code> .
Claude Code does not recognise <code>/speckit-...</code>	Either you started outside the project folder, or you are typing a dot instead of a hyphen.
A message tells you to run <code>/speckit.something</code>	Known defect: translate the dot into a hyphen.
<code>command not found: claude</code>	Not on the PATH. New terminal and <code>claude doctor</code> .
Authentication errors	<code>/status</code> : it is nearly always <code>ANTHROPIC_API_KEY</code> left unset.

Common problems (2/2)

Symptom	What to do
Unexpected files appear in <code>specs/</code>	That is normal: each phase writes its own artefacts.
The spec comes out full of technical detail	Your prompt included it. Redo it talking only about behaviour.
The spec is huge and generic	Too much scope. One feature per specification.
<code>implement</code> takes forever	<code>Esc</code> and relaunch naming concrete tasks.
The code does not match the plan	<code>/speckit-converge</code> : it compares and adds what is missing.
The agent ignores the constitution	Principles that are not verifiable, or too many of them. Trim it.
Your neighbour gets a different result	That is expected. Which is why the artefacts get reviewed.

Reference: command line

```
# installation with a pinned version
uv tool install specify-cli \
  --from git+https://github.com/github/spec-kit.git@v1.0.2

# new project
specify init <name> --integration claude
specify init --here --force --integration claude

# the repository is NOT created for you since v0.10.0
git init
specify extension add git

specify integration list
specify self check
specify self upgrade [--dry-run] [--tag vX.Y.Z]
```

Reference: inside the agent

/speckit-constitution	project principles	
/speckit-specify	what to build and why	
/speckit-clarify	resolve ambiguities	(before plan)
/speckit-plan	how to build it	
/speckit-checklist	requirements quality	
/speckit-tasks	in which order	
/speckit-analyze	consistency	(before implement)
/speckit-implement	build	
/speckit-converge	reconcile code and artefacts	
/speckit-git-commit	commit; offered before implement	
/speckit-git-feature	create the feature branch	
/speckit-git-initialize	initialise the repository	

Reference: Claude Code

```
claude          # open a session in the current directory
claude --version
claude doctor    # installation diagnostics

/status         which endpoint and token it connects with
/help          available commands
/clear         empty the context
/logout        drop a stored Anthropic session
/exit          quit
Esc            interrupt the agent
Shift+Tab      change permission mode
```

Golden rules

- The specification describes observable behaviour; the plan describes technology. Do not mix them.
- One feature, one folder under `specs/`.
- `clarify` before `plan`; `analyze` before `implement`.
- If a requirement has no number, it is not a requirement yet.
- Facing a change, edit the highest affected artefact and regenerate downwards.
- Commit the artefacts separately from the code.
- Manual mode while you generate artefacts; accept edits while you implement.

If you use another agent

Agent	Invocation
Claude Code	Skills with a hyphen: <code>/speckit-specify</code> , in <code>.claude/skills/</code>
Command-based agents	With a dot: <code>/speckit.specify</code> , as in the official documentation
GitHub Copilot (editor)	Installs skills by default. For commands, initialise with <code>--integration-options</code>
Copilot CLI	Selected with <code>/agents</code>
Codex CLI, ZCode	<code>\$speckit-specify</code>
Kimi and similar	<code>/skill:speckit-specify</code>

`specify integration list` overrules any tutorial.
The method, the prompts and the artefacts are identical in all of them.